CLASSES

CL217 – Object Oriented Programming

5.1 BASIC CONCEPTS OF OBJECT PROGRAMMING

In Object-Oriented Programming (OOP), the concept of "mapping the real-world model" refers to representing real-world entities and their behaviors within a program using classes, objects, and other OOP principles

Object-Oriented Programming (OOP)

- **OOP** maps **real-world entities** into **software objects** by representing their properties as data and their behaviors as methods.

- **OOP** is a programming paradigm that organizes software design using objects and classes instead of functions and logic.
  - Objects represent real-world entities
  - Objects contain data + behavior
  - Programs are built using interacting objects.
  - Improves modularity, reusability, and maintainability

- **Examples:**
  - Car, Student, BankAccount

Encapsulation (Bundling)

- Data encapsulation is the process of combining **data members (variables)** and **member functions (methods)** into a **single unit** called a **class**.
- **Bundling data (variables)** and **functions (methods)** together into a single unit called a class. It binds data and functions together.
  - Wraps data and methods together
  - Keeps related code in one place
  - Improves organization and security
- **Example**:

```
class Student {
        string name;
        string getName();
};
```

Data Hiding

- **Data hiding** is the technique of **restricting direct access** to the internal data of a class using **access specifiers** like **private** and **protected**.
  - Prevents unauthorized access
  - Protects data integrity
  - Access through public methods/ interfaces only
  - Data is protected from outside access

- **Example:**

```
class Account {
        private:
        double balance;
};
```

Abstraction

- **Abstraction** is the process of showing only essential features while **hiding internal implementation details**.
  - Reduces complexity
  - Focus on what the object does
  - Implementation details are hidden
- **Example:**

```
class Car {
        void start() {// complex engine logic hidden}
};
int main() {
        Car c;
        c.start(); // Simple call, user only start() without knowing
                   // the internal engine mechanism.
}
```

Class

- A **class** is a **key building block of OOP**. Just like a **blueprint** guides the construction of a house, a **class** defines the structure and behavior of objects, ensuring data and functionality are properly encapsulated and organized.

- A **class** is a **template** or **layout** that defines the **structure (attributes)** and **behavior (methods)** of **objects**.

- A **class encapsulates data and functions** into a **single unit**, ensuring related information and behavior are organized and protected.

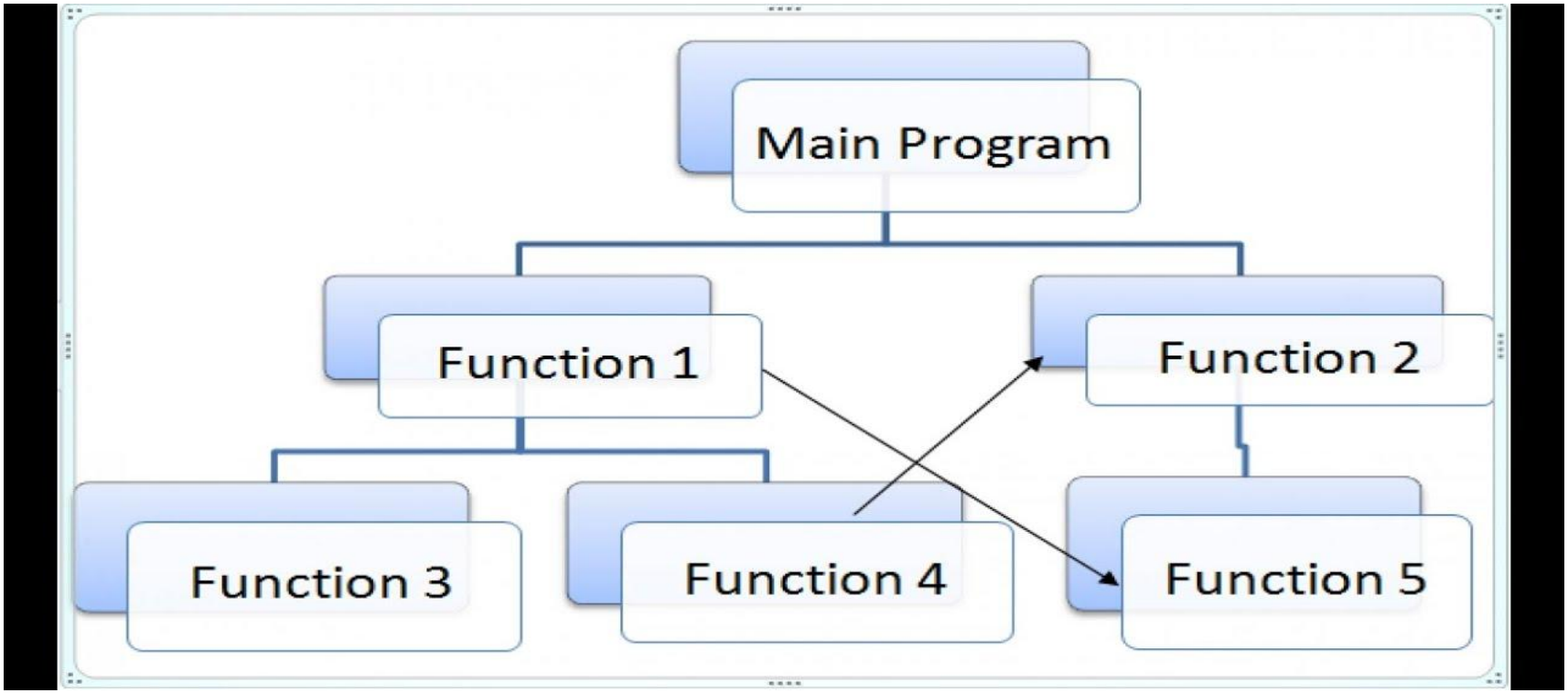
Class

- It acts as a **generalized design** or **plan** from which **multiple objects (instances)** can be created.
- In short, a **class models real-world entities in software** by **combining attributes** and **methods** while hiding unnecessary details.

- **Real-World Analogies/ Examples**
  - **Blueprint:** A house blueprint guides construction; each house built is an object.
  - **Mould:** A toy mould shapes plastic; each toy produced is an object.
  - **Die:** A coin die stamps metal; each coin is an object.
  - **Stamp:** A seal stamp creates impressions; each imprint is an object.

5.1.1 Classes and objects in real life

- All of our programs and all the techniques we've used up till now fall into the so-called **procedural style of programming.**



5.1.1 Classes and objects in real life

- Procedural programming works very well for simple projects and not large ones.
- Large and complex projects carried out by large teams consisting of many developers use Object approach .
- C++ language was created as a universal tool for object programming.

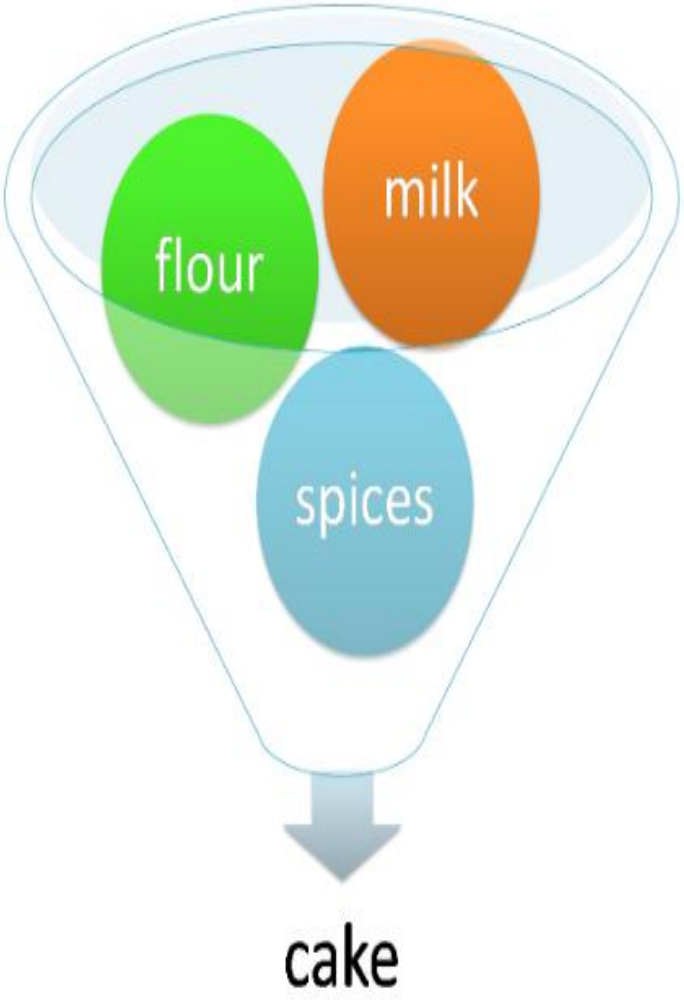
5.1.2 Classes and objects in real life

- In the **procedural** approach, we can distinguish two different and separate worlds:
  - **data:** populated by variables of different types
  - **code:** inhabited by code grouped into functions.

- The **object** approach suggests a completely different way of thinking.
  - **The data and the code are enclosed together in the same world, divided into classes**

5.1.2 Classes and objects in real life

- Every **class** is like a recipe that can be used when you want to create a useful object
- You may produce as many objects as you need to solve your problem.
- Every **object** has a set of **properties** and is able to perform a set of activities (**operations, methods**).



5.1.2 Classes and objects in real life

- The recipes may be modified if they're inadequate for specific purposes and, in effect, new classes may be created.
  - **The new classes inherit properties and methods from the originals and usually add some new ones, creating new and more specific tools.**
- **Objects** are **materializations** of ideas expressed in **classes**
  - much like a **piece of cheesecake** on your plate is an incarnation of an idea expressed in the **recipe** printed in an old cookbook.

Example: Compute rectangle area.

```
class rectangle {
private:
        int length;
        int width;
public:
        void set_len(int a) {
        length=a; }
        void set_wid(int a) {
        width=a; }
        int get_len() {
        return length; }
        int get_wid() {
        return width; }
        int area() {
        return width*length;}
};
```

```
int main()
{
int L, W;
rectangle aobject;
cout<<"Enter the length and width of rectangle(L,W)";
cin>>L>>W;
aobject.set_len(L);
// illegal  aobject.length=L; //because, length is private
aobject.set_wid(W);
// illegal  aobject.width=W; //because width is private
cout<<"Length:"<<aobject.get_len()<<endl;
cout<<"Width:"<<aobject.get_wid()<<endl;
cout<<"Area: "<<aobject.area()<<endl;
return 0;
}
```

5.1.3 Class – what is it?

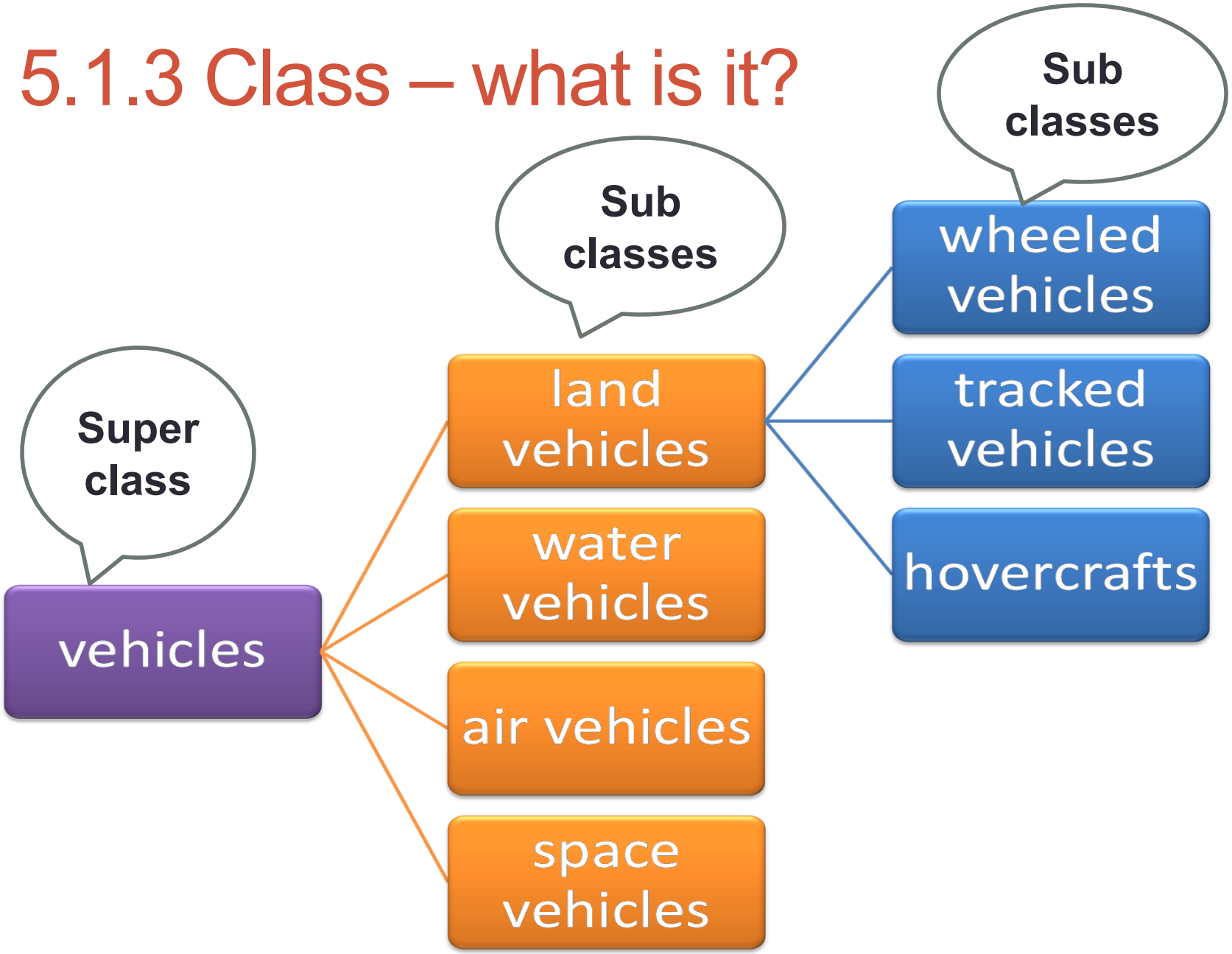
- Example of Classes (vehicles).
  - All vehicles existing (and not yet existing) in the world are related to each other by a single, important feature: **the ability to move**.
  - We suggest the following definition for any member of the class vehicles:

    vehicles are artificially created entities used for transportation, moved by forces of nature and directed (driven) by humans.

5.1.3 Class – what is it?

- The “vehicles” class is very broad. Too broad.
  - We need to define some more **specialized** classes.
  - The specialized classes are (will be) called **“sub-classes**”.
  - The “vehicles” class will be a “**super-class**” for them all.
  - The most general and the widest class is always at the top (a super) while its descendants are below (subs).

5.1.3 Class – what is it?



5.1.5 Object – what is it?

- Object is an instance of class, which holds the data variables declared in class and the member operations (functions) work on these class objects.
- You can create as many objects as you want from the same class.

5.1.5 Object – what is it?

- For example:
  - any personal car is an object that belongs to the “wheeled vehicles” class.
  - It also means that the same car belongs to all the superclasses of its home class; therefore, it’s a member of the “vehicles” class, too.
- Each subclass is more specialized (or more specific) than its superclass.
- Each superclass is more general (more abstract) than all its subclasses.

5.1.5 Object – General vs Specific

- General categories are like broad groups in the real world, and specific categories are the detailed subgroups.

```
LivingThing
 ├─ Plants
 │   ├─ Rose
 │   └─ Tulip
 └─ Animals
     ├─ Herbivores (eat plants)
     │   └─ Cow
     ├─ Carnivores (eat meat)
     │   ├─ BigCats
     │   │   └─ Lion
     │   └─ DomesticCat
     └─ Omnivores (eat plants & meat)
         └─ Bear
```

5.1.7 What does any object have?

- The object programming convention assumes that every existing object may be equipped with three groups of attributes:
  - A **name** that uniquely identifies it
  - A set of **individual properties (attributes)**
  - A set of **abilities** to perform specific **activities** that can change the object itself (**operations**)
-

5.1.7 What does any object have?

- There's a hint that can help you identify any of these three spheres. Whenever you describe an object and you use:
  - a **noun**, you probably define the **object's name**
  - **adjective**, you probably define the object's **property**
  - a **verb**, you probably define the object's **activity**

5.1.7 What does any object have?

- Example 1:

"Max is a large cat who sleeps all day"

Object name = Max

Home class = Cat

Property = Size (large)

Activity = Sleep (all day)

- Example 2:

"A pink Cadillac went quickly"

Object name = Cadillac

Home class = Wheeled vehicles

Property = Colour (pink)

Activity = Drive (quickly)

5.1.8 Why all this?

- The class you define has nothing to do with object creation
  - The class itself isn't able to create an object – you have to create it yourself.
- It's now time to show you how to define the simplest class and how to create an object.

```
class  OurClass{ };
```

- The definition begins with the keyword **class**.
- The keyword is followed by an **identifier** that names the class
- Next, you add a pair of **curly brackets**.
  - The content inside these define all the class properties and activities.
  - **Our curly brackets are empty, so the class is empty too.**

5.1.9 The very first object

- The newly defined class becomes an equivalent of a ***data type***, and we can use it as a type name.
- Imagine that we want to create one object of the ***`OurClass`*** class.
- We declare a variable that can store objects of that class and create an object at the same time.

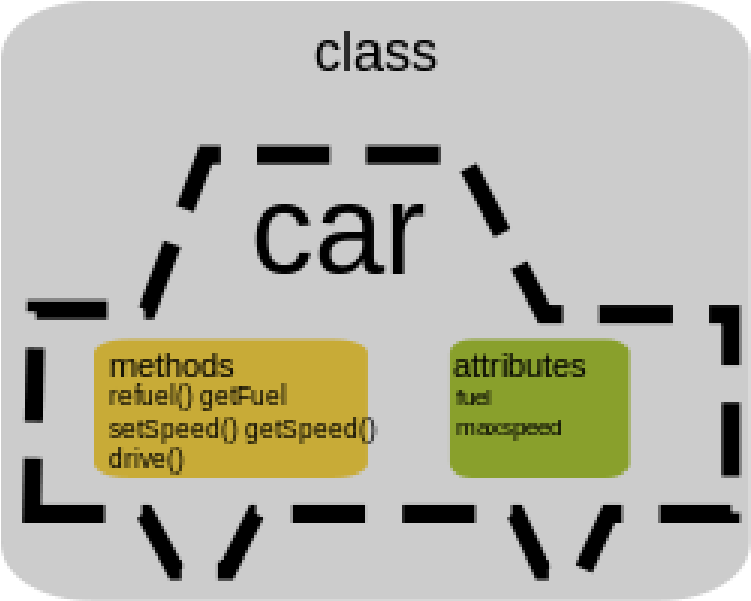
```
OurClass ourObject;
```

5.3 ANATOMY OF THE CLASS

5.3.1 Class components

- A class is an **aggregate** consisting of **variables** (also called fields or properties) and **functions** (sometimes called operations/methods).
- Both variables and functions are class **components /members**.

```
class classIdentifier
{
    classMembersList
};
```



5.3.2 Access specifier

- Three categories of class members
  - `private` (**default**)
    - Member cannot be accessed outside the class
  - `Public`
    - Member is accessible outside the class
  - `Protected`
    - Used with inheritance (next chapter)

Any member (variable or function) can be public, private or protected

5.3.2 Access specifiers

- The following class has three components:
  - one variable of type ***int*** called ***value***
  - two functions called ***setVal*** and ***getVal*** respectively.
  - class is named *Class*.

```
class Class {
  int value;
  void setVal(int value);
  int getVal(void);
};
```

- Since all the components are declared without the use of an **access specifier,** all three components are **private**

5.3.2 Access specifiers

- The ***setVal*** and ***getVal*** components are **public** – they're accessible ***to all users of the class***.
- The *value* component is **private** – it's accessible only ***within the class***.

```
class Class {
    public:
        void setVal(int value);
        int getVal(void);
    private:
        int value;
};
```

5.3.3 Creating an object

- Any object of the class is equipped with all the components defined in the class.

```
Class the_object();
```

- The **public** components are available for use. You can do this:

```
the_object.setVal(0);
```

- The **private** components are hidden and unavailable. You mustn't do this:

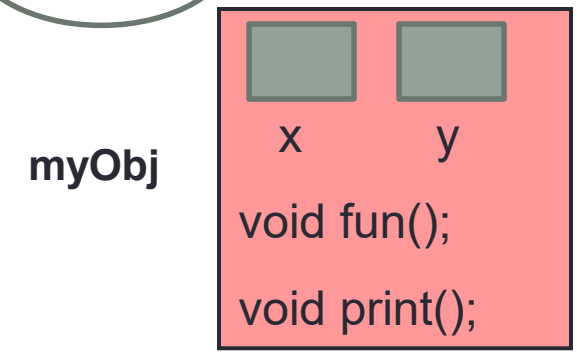
```
the_object.value = 0; //Syntax Error
```

Example:

```
#include <iostream>
using namespace std;

class A
{
    int x;
    int y;
public:
    void print(){cout<<x<<"    "<<y<<endl;}
    void fun();
};

void A::fun()
{
    A obj;
    obj.x=5;
    obj.y=10;
    obj.print();
}
int main()
{
A myObj;
myObj.fun();
}
```

We can define the function member inside the class

We can define the function member outside the class using ***:: operator***



Example

```
class clockType
{
public:
    void setTime(int, int, int);
    void getTime(int&, int&, int&) const;
    void printTime() const;
    void incrementSeconds();
    void incrementMinutes();
    void incrementHours();
    bool equalTime(const clockType&) const;

private:
    int hr;
    int min;
    int sec;
};
```

These functions cannot modify the member variables of type `clockType`

`const`: formal parameter can't modify the value of the actual parameter

`private` members, can't be accessed from outside the class

Class Functions

- Getter function: member function that only accesses the value(s) of member variable(s)
- Setter function: member function that modifies the value(s) of member variable(s)
- Constant function:
  - Member function that cannot modify member variables
  - Use `const` in function heading

```
#include <iostream>
using namespace std;

class A
{
private:
     int x;
     int y;
public:
     int fun1(const int& a, int& b);
     void fun2(int& a, int b) const;
     void print();
};
int A::fun1(const int & v1, int & v2)
{
     //v1=x;  illegal
     x= v1;
     y= v2;
     return x+y;
}
void A::fun2(int& v1, int v2) const
{
     //x=v1 illegal
     v1= x;
     v2= y;
}
void A::print()
{
     cout<<x<<"    "<<y<<endl;
}

void main ()
{
     A obj1;
     int var1=3, var2=6, var3=10;
     var1=obj1.fun1(var2, var3);
     cout<<var1<<"    "<<var2<<"    "<<var3<<endl;
     obj1.print();
     obj1.fun2(var3, var2);
     cout<<var1<<"    "<<var2<<"    "<<var3<<endl;
}
```

5.3.4 Overriding component names

- A function that is a class component has full access to other class components, including `private` ones.
- If any function introduces an entity of the name identical to any of the class components, the name of the class component is **overridden**.
- It can be accessed only by the qualification with the home class name.

Example

```
class A{
    int x;
public:
    void setX(int x)
    {x = x;}
    void print()
    {cout<<x<<endl;}
};
void main()
{
    A obj;
    obj.setX(10);
    obj.print();
}
```

Rubbish value

```
class A{
    int x;
public:
    void setX(int x)
    {A::x = x;}
    void print()
    {cout<<x<<endl;}
};
void main()
{
    A obj;
    obj.setX(10);
    obj.print();
}
```

10

5.3.4 Overriding component names

- The *setX* function uses a parameter called *x*.
- The parameter overrides the class component called *x*. This means that the assignment

```
x = x;
```

applies to the value parameter and has nothing to do with the component of the same name.

- One of the proper forms of the assignment looks like this:

```
A::x = x;
```

- The qualification unveils the overridden name of the component.

5.3.6 Qualifying component names (1)

- If any class function body is given outside the class body, its name must be qualified with the home class name and the "**::**" operator.
- A function defined in this way has the same full access to all class components as any function defined inside the class.
- All rules for **overriding** names are valid too.

5.3.6 Qualifying component names (1)

```
class Class {
public:
   void setVal(int value) {
         this -> value = value;
   }
   int getVal(void);
private:
   int value;
};

int Class::getVal(void) {
     return value;
}
```

5.3.7 Qualifying component names (2)

- Class function names **may be overloaded** just like ordinary function names. The default parameter values may be used too.

- All the other restrictions on the construction of the overloaded class functions apply, too.

Example

```
class Class {
public:
        void setVal(int value) { this -> value = value;  }
        void setVal(void) { value = -2; }
        int getVal(void) { return value; }
private:
        int value;
};
int main()
{  Class obj;
    obj.setVal(20); // call the first version of the function
    obj.setVal(); // call the second version of the function
}
```

5.3.8 Constructors

- **A function with a name identical to its home class name is called a constructor**.
- The constructor is intended **to construct the object during its creation,** i.e., to initialize field values, allocate memory, create other objects, etc.
- The constructor may access all object components like any other class member function, but should not be invoked directly.
- **A constructor has no type:** This means it does not return anything, not even void, because its sole purpose is to initialize the object.

Example

```
class Class {
public:
        Class(void) { this -> value = -1; }
        void setVal(int value) { this -> value = value; }
        int getVal(void) { return value; }
private:
      int value;
};
```

5.3.8 Constructors

- Declaring the object of the class, e.g. by doing it in the following way:

```
Class object;
```

implicitly invokes the constructor.

- Note
  - you're not allowed to do either this:

    ~~object.Class()~~

  - or this

    ~~Class::Class();~~

```
#include <iostream>
using namespace std;

class A
{
    int x;
    int y;
public:
    void print(){cout<<x<<"    "<<y<<endl;}
    A(){x=10; y=20; cout<<"Constructor"<<endl;}
};


int main()
{
A myObj, yourObj;
myObj.print();
yourObj.print();
}
```

5.3.9 Overloading constructor names (1)

- Constructors may be overloaded too, depending on specific needs and requirements.
- The following class has two different constructors that differ in their number of parameters.

- The actual constructor is chosen during the object's creation.

```
class Class {
    public:
        Class(void) { this -> value = -1; }
        Class(int val) { this -> value = val; }
         void setVal(int value) { this -> value = value; }
        int getVal(void) { return value; }
    private:
      int value;
};
```

5.3.9 Overloading constructor names (1)

- Look at the following snippet, please:

```
Class object1, object2(100);
cout << object1.getVal() << endl;
cout << object2.getVal() << endl;
```

- The *object1* object will be created using a parameter-less constructor, while the *object2* will be created with a one-parameter constructor.

- The snippet will output the following values:

  -1

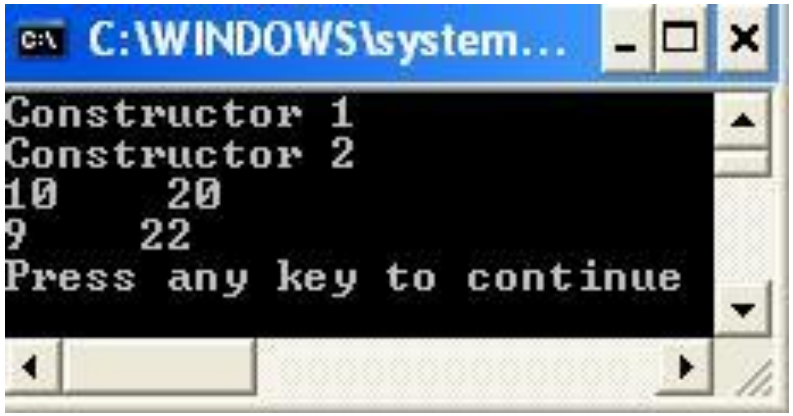
  100

5.3.9 Overloading constructor names (1)

- Note that **you mustn't demand the use of a non-existent constructor** – this means that the following snippet is wrong:

```
Class objectx(2,100);
```

```
#include <iostream>
using namespace std;

class A
{
    int x;
    int y;
public:
    void print(){cout<<x<<"    "<<y<<endl;}
    A(){x=10; y=20; cout<<"Constructor 1"<<endl;}
    A(int, int);
};
A::A(int a, int b)
{x=a; y=b; cout<<"Constructor 2"<<endl;}

int main()
{
A myObj, yourObj(9, 22);
myObj.print();
yourObj.print();
//yourObj.A(5,7); error
}
```



5.3.10 Overloading constructor names (2)

- If a class has a constructor (or more precisely, at least one constructor), **one of them must be chosen** during object creation
  - i.e. you're not allowed to write a declaration which doesn't specify a target constructor.

```
class Class {
public:
        Class(int val) { this -> value = val; }
        void setVal(int value) { this -> value = value; }
        int getVal(void) { return value; }
private:
     int value;
};
Int main()
{
   Class obj1(5);  //legal statement
   Class obj2; // Syntax Error (no default constructor)
}
```

Constructors and Default Parameters

```
clockType clockType(int = 0, int = 0, int = 0);    //Line 1
```

- Using **`clockType`** constructor, you can declare **`clockType`** objects with zero, one, two, or three arguments as follows:

```
clockType clock1;                    //Line 2
clockType clock2(5);                 //Line 3
clockType clock3(12, 30);            //Line 4
clockType clock4(7, 34, 18);         //Line 5
```

Note: if you have a constructor with all default parameters you cannot declare a constructor without parameters, they both called default constructor.

```
clockType  clockType(int = 0,  int = 0,  int = 0);

clockType clockType ();    //illegal in the same class
```

5.3.13 Destructors

- Destructors are functions without any type
- The name of a destructor is the character '~' followed by class name
  - For example:

```
~clockType();
```

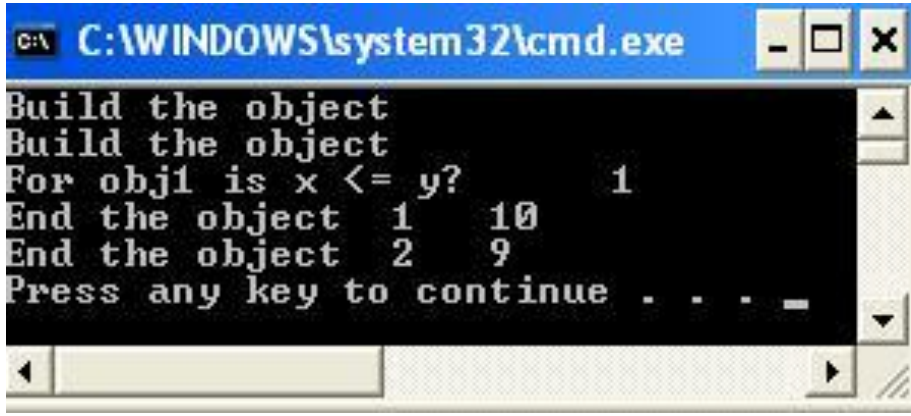
- A class can have only one destructor
  - The destructor has no parameters
- The destructor is automatically executed when the class object goes out of scope

Example

```
#include <iostream>
#include <string>
using namespace std;

class A
{
public:
    A() { cout<<"Welcome\n";}
    ~A() {cout<<" Good bye\n";}
};

void fun()
{ A var2; cout<< "Out of fun " ;
}

void main()
{
    A var1;
    fun();
    fun();
    cout<<"Out of main";

}
```

Example

```
#include <iostream>
using namespace std;;

class A
{
     int x;
     void privateFun();
public:
     int y;
     bool publicFun(){privateFun();     return (x<=y) ;}
     A();
     ~A(){cout<<"End the object  "<<x<<"    "<<y<<endl;}
};
void A::privateFun()
{
     x++;
     y--;
}
A::A()
{
     x=1;
     y=10;
     cout<<"Build the object  "<<endl;
}
void main()
{
     A obj1, obj2;
     cout<<"For obj1 is x <= y?       "<<obj1.publicFun()<<endl;
}
```



Arrays of Class Objects (Variables) and Constructors

- If a class has constructors and you declare an array of that class's objects, the class should have the default constructor

```
clockType arrivalTimeEmp[100];
```

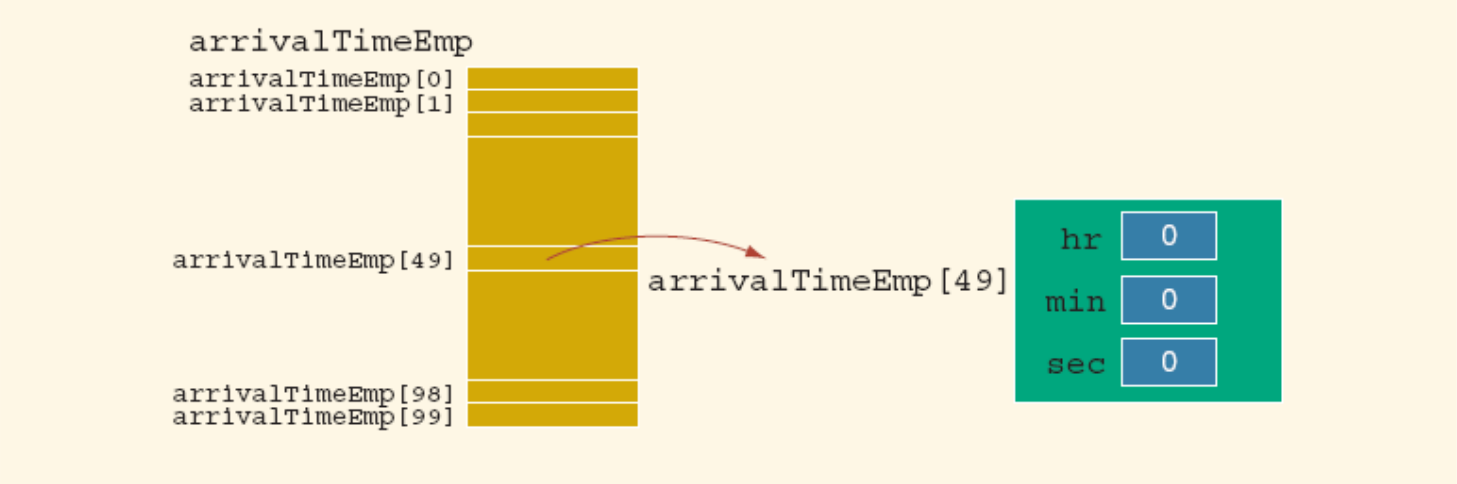


FIGURE 12-9 Array `arrivalTimeEmp`

Arrays of Class Objects (Variables) and Constructors (continued)

```
arrivalTimeEmp[49].setTime(8, 5, 10);
```

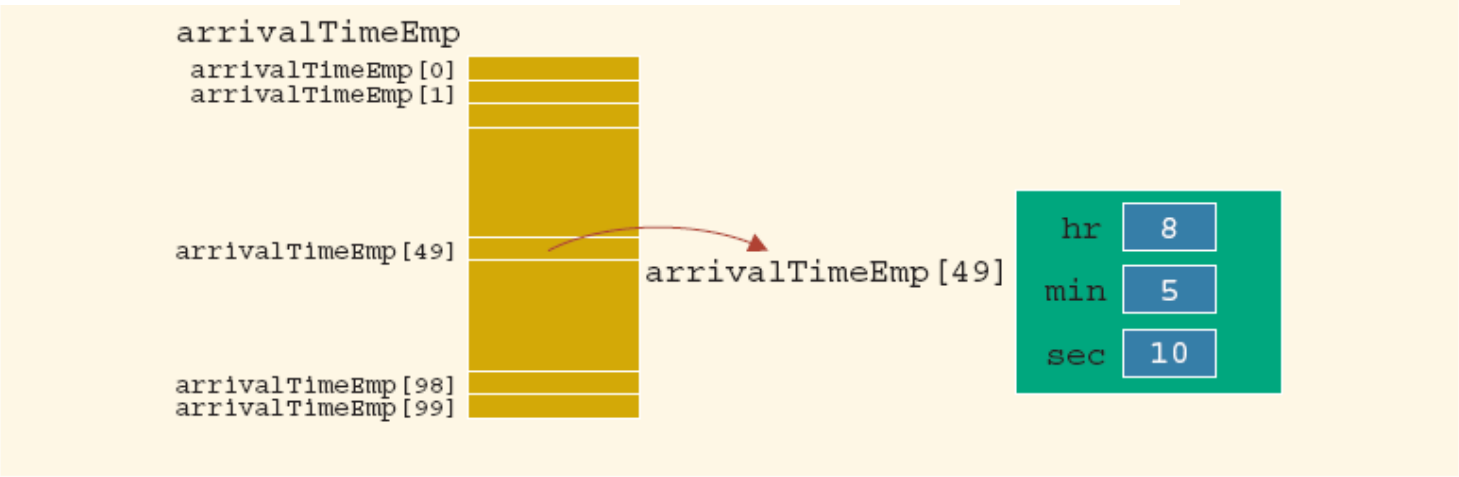


FIGURE 12-10 Array `arrivalTimeEmp` after setting the time of employee 49

```
for (int j = 0; j < 100; j++)                    //Line 3
{
    cout << "Employee " << (j + 1)
         << " arrival time: ";
    arrivalTimeEmp[j].printTime();               //Line 4
    cout << endl;
}
```

Example

```
class Employee{
      string name;
      int id;
      double salary;
public:
      Employee(string="",int=0,double=0);
      void setAll(string,int,double);
      double getSalary()const;
      void setSalary(double s);
      void print()const;
      ~Employee();
};
```

Example

```
Employee::Employee(string n,int i,double s)
{
      setAll(n,i,s);
}
void Employee::setAll(string n,int i,double s)
{
      name = n;
      id = i;
      setSalary(s);
}
double Employee::getSalary()const
{  return salary; }
```

Example

```
void Employee::setSalary(double s)
{
   if( s >= 100 && s <= 1000)
            salary = s;
      else if( s < 100)
            salary = 100;
      else
            salary = 1000;
}
void Employee::print()const
{
      cout<<"Employee name: "<<name<<" Id = "<<id<<"
Salary = "<<salary<<endl;
}
Employee::~Employee()
{
      cout<<"goodbye "<<name<<endl;}
```

Example

/*Write a NON-MEMBER function that receives an object of type employee and the increasing percentage. This function should update the salary for the employee by the parameter percentage.*/

```
void updateSalary(Employee &e,double p)
{
      double s = e.getSalary();
      s = s + s*p;
      e.setSalary(s);
      //e.setSalary(e.getSalary()+e.getSalary()*p);
}
```

Example

/*write a non-member function that receives an array of employee and print all information its elements.
*/

```
void print(Employee e[],int size)
{
      for(int i = 0; i < size; i++)
            e[i].print();
}
```

```
void main()
{
//create an object of type employee with the
following info: Ali,123,700
      Employee admin("Ali",123,700);
//create an array of type employee of size 3
      Employee emp[3];
//prompt the user to enter the information of all
employees in the array
      string n;
      int id;
      double s;
      for(int i = 0; i < 3; i++)
      {
            cin>>n>>id>>s;
            emp[i].setAll(n,id,s);
      }
```

```
//print all employee in this program
     admin.print();
     for(int i = 0; i < 3; i++)
          emp[i].print();
//Increase the salary for the employee admin by 30%
     updateSalary(admin,.3);
//Increase the salary for the third employees in the
array by 5%
     updateSalary(emp[2],.05);
//print the information of the emp array
      print(emp,3);
}
```

Data Abstraction, Classes, and Abstract Data Types

- Abstraction
  - Separating design details from usage
  - Separating the logical properties from the implementation details
- Abstraction can also be applied to data
  - Abstract data type (ADT): data type that separates the logical properties from the implementation details

Data Abstraction, Classes, and Abstract Data Types (continued)

EXAMPLE 12-8

A list is defined as a set of values of the same type. Because all values in a list are of the same type, a convenient way to represent and process a list is to use an array. You can define a list as an ADT as follows:

```
dataTypeName
    listType
domain
    Every listType value is an array of, say, 1000 numbers
operations
    Check to see whether the list is empty.
    Check to see whether the list is full.
    Search the list for a given item.
    Delete an item from the list.
    Insert an item in the list.
    Sort the list.
    Destroy the list.
    Print the list.
```

Data Abstraction, Classes, and Abstract Data Types (continued)

```
class listType
{
public:
    bool isEmptyList() const;
    bool isFullList() const;
    int search(int searchItem) const;
    void insert(int newElement);
    void remove(int removeElement);
    void destroyList();
    void printList() const;
    listType(); //constructor

private:
    int list[1000];
    int length;
};
```

A `struct` versus a `class`

- By default, members of a `struct` are `public`
  - `private` specifier can be used in a `struct` to make a member private
- By default, the members of a `class` are `private`
- `class`es and `struct`s have the same capabilities

A `struct` versus a `class` (continued)

- In C++, the definition of a `struct` was expanded to include member functions, constructors, and destructors
- If all member variables of a `class` are `public` and there are no member functions
  - Use a `struct`